

RAPPORT TECHNIQUE

Analyse détaillée d'un projet open-source

STRIX

« Des hackers IA open-source pour trouver et corriger les vulnérabilités de vos applications »

Plateforme autonome de tests d'intrusion fondée sur des agents IA (LLM)

Dépôt : github.com/usestrix/strix

Paquet PyPI : [strix-agent](#) · Licence : Apache 2.0

Éditeur : Strix AI · Documentation : docs.strix.ai

Rapport rédigé le 18 juillet 2026

Résumé

Strix est une plateforme open-source (licence Apache 2.0) de tests d'intrusion automatisés reposant sur des agents d'intelligence artificielle fondés sur de grands modèles de langage (LLM). À la manière de véritables attaquants, ces agents exécutent dynamiquement le code de la cible, découvrent des vulnérabilités et les valident au moyen de preuves de concept (PoC) réelles, réduisant ainsi les faux positifs des analyses statiques. Le présent rapport analyse en profondeur l'architecture du projet — un orchestrateur en ligne de commande côté hôte, un bac à sable Docker exécutant les outils, et une intégration LLM via LiteLLM —, son système multi-agent (« graphe d'agents »), sa boîte à outils offensive (proxy HTTP, navigateur, terminal, runtime Python, reconnaissance), sa base de connaissances (skills), son arsenal d'outils Kali, son flux d'exécution et ses livrables, ainsi que ses modalités d'installation et d'intégration CI/CD. Le rapport conclut sur les forces, les limites et le positionnement de Strix dans l'écosystème du pentest assisté par IA.

Mots-clés :

Strix, tests d'intrusion, IA agentique, LLM, graphe d'agents, Docker, Kali, PoC, DevSecOps.

Table des matières

Clic droit → « Mettre à jour les champs » pour générer la table.

Liste des figures

- Figure 2.1 — Architecture technique de Strix (hôte, bac à sable, LLM, sorties)
- Figure 3.1 — Graphe d'agents : orchestration multi-agent
- Figure 4.1 — Catégories d'outils des agents
- Figure 5.1 — Classes de vulnérabilités couvertes (regroupées)
- Figure 7.1 — Flux d'exécution d'une campagne

Introduction

La sécurité offensive — au premier rang de laquelle le test d'intrusion — reste indispensable pour évaluer la résilience réelle d'une application. Réalisée manuellement, elle est toutefois lente, coûteuse et tributaire de l'expertise de l'opérateur ; automatisée par des outils classiques, elle génère de nombreux faux positifs. L'essor des grands modèles de langage et des architectures « agentiques » a fait émerger une nouvelle catégorie d'outils capables de planifier et d'exécuter des actions offensives de façon autonome.

Strix s'inscrit dans cette dynamique. Projet open-source publié sous licence Apache 2.0 et distribué via PyPI (strix-agent), il propose des agents IA qui « se comportent comme de vrais hackers » : ils exécutent dynamiquement le code, découvrent des vulnérabilités et les valident par des preuves de concept. Ce rapport en présente une analyse technique complète, de l'architecture aux livrables, afin d'en comprendre le fonctionnement, les capacités et les limites.

Chapitre 1 — Présentation générale de Strix

1.1 Vision et principe

Strix (« hiboux/rapaces IA ») repose sur un principe simple : confier à des agents autonomes, pilotés par un LLM, la conduite d'un test d'intrusion de bout en bout. Contrairement à un scanner de vulnérabilités traditionnel, un agent Strix raisonne, planifie, exécute des outils réels dans un environnement isolé, observe les résultats et itère — jusqu'à démontrer concrètement l'exploitabilité d'une faille.

1.2 Capacités clés

- **Boîte à outils complète** — un arsenal offensif prêt à l'emploi (proxy, navigateur, terminal, Python, reconnaissance).
- **Équipes d'agents** — des agents qui collaborent et passent à l'échelle (graphe d'agents).
- **Validation réelle** — des preuves de concept plutôt que des faux positifs.
- **Approche « developer-first »** — une CLI avec des rapports exploitables.
- **Correction et reporting** — génération de rapports et aide à la remédiation.

1.3 Cas d'usage

- **Sécurité applicative** — détection et validation de vulnérabilités critiques.
- **Pentest rapide** — des tests en heures plutôt qu'en semaines, avec rapports de conformité.
- **Bug bounty** — automatisation de la recherche et génération de PoC.
- **Intégration CI/CD** — exécution à chaque pull request pour bloquer le code vulnérable.

1.4 Écosystème et distribution

Le projet est publié sous licence Apache 2.0 sur GitHub (usestrix/strix) et distribué via PyPI sous le nom strix-agent. Il s'accompagne d'une documentation (docs.strix.ai), d'une offre hébergée (app.strix.ai) et d'intégrations GitHub Actions. Une communauté anime le projet (Discord, X).

Chapitre 2 — Architecture technique

2.1 Vue d'ensemble

L'architecture de Strix distingue clairement l'orchestration (côté hôte) de l'exécution des outils (dans un bac à sable isolé). Trois éléments principaux collaborent : un CLI Python qui orchestre les agents et présente une interface en terminal (TUI Textual) ; un bac à sable Docker fondé sur une image Kali qui exécute les outils via un serveur d'outils ; et un fournisseur LLM interrogé à travers la bibliothèque LiteLLM.

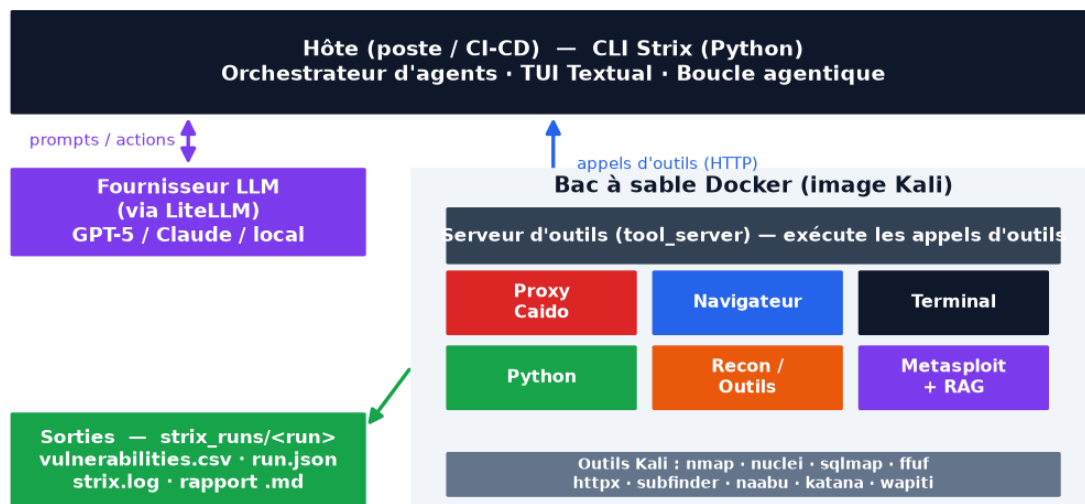


Figure 2.1 — Architecture technique de Strix (hôte, bac à sable, LLM, sorties).

2.2 L'orchestrateur (hôte)

Le CLI Strix, écrit en Python, constitue le cerveau du système. Il gère la boucle agentique — envoi du contexte au LLM, réception des actions, invocation des outils, réintégration des résultats — et coordonne le graphe d'agents. Il expose une interface interactive en terminal (TUI) fondée sur Textual, ainsi qu'un mode non interactif (headless) pour l'automatisation.

2.3 Le bac à sable (Docker/Kali)

Toutes les actions offensives s'exécutent dans un conteneur Docker isolé, bâti sur une image de type Kali Linux contenant l'arsenal d'outils. Un « serveur d'outils » (`tool_server`) tourne dans le conteneur et reçoit, via HTTP, les appels d'outils émis par l'hôte. Cette isolation protège le poste de l'opérateur et garantit un environnement reproductible ; l'image du bac à sable est téléchargée automatiquement au premier lancement.

2.4 L'intégration LLM

Strix s'appuie sur LiteLLM pour rester agnostique vis-à-vis du fournisseur : OpenAI (GPT-5), Anthropic (Claude), ou des modèles locaux. Le module LLM gère notamment la compression de mémoire (pour les longs contextes) et la déduplication, afin de maîtriser le coût et la taille des échanges.

2.5 Runtime et exécution

La couche runtime (`docker_runtime`, `tool_server`) démarre le conteneur, attend la disponibilité du serveur d'outils (contrôle de santé) puis relaie les appels d'outils. Les résultats (vulnérabilités, journaux, télémétrie) sont persistés côté hôte dans un répertoire de campagne.

Chapitre 3 — Le système multi-agent (graphe d'agents)

3.1 Orchestration multi-agent

La force de Strix réside dans son « graphe d'agents » : plutôt qu'un agent unique, un agent racine décompose la cible en tâches indépendantes et engendre dynamiquement des sous-agents spécialisés (reconnaissance, injection, contrôle d'accès, SSRF/XXE, reporting...). Ces agents s'exécutent en parallèle, collaborent et partagent leurs découvertes.

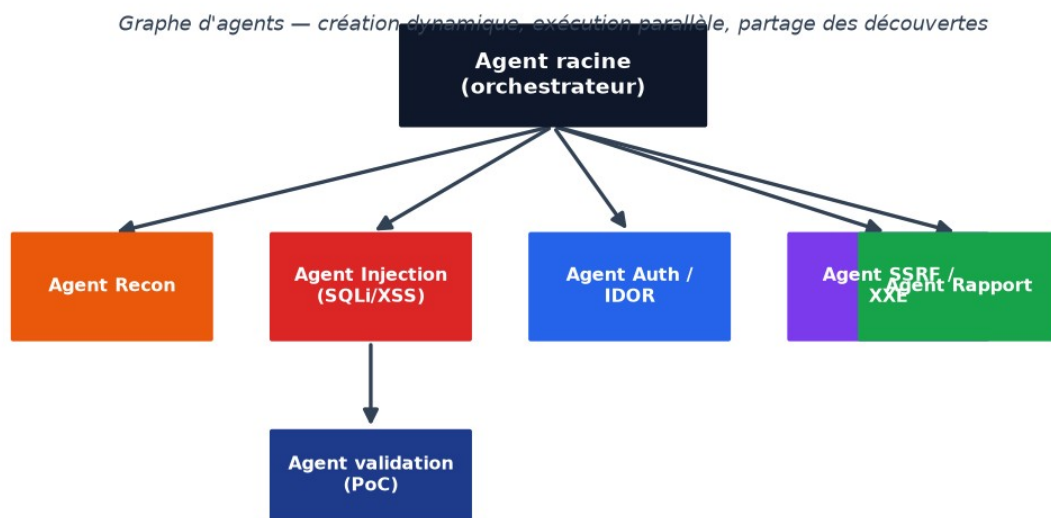


Figure 3.1 — Graphe d'agents : orchestration multi-agent.

3.2 L'agent Strix et son prompt

Chaque agent est une instance de StrixAgent, configurée par un prompt système (modèle Jinja) définissant les règles, l'environnement, les phases obligatoires et les tactiques d'efficacité. Un agent peut charger jusqu'à cinq « skills » spécialisés pertinents pour sa tâche.

3.3 Coordination et passage à l'échelle

La coordination privilégie l'indépendance des tâches et l'exécution parallèle. Des sous-agents de validation confirment l'exploitabilité par PoC, et des agents de reporting documentent les découvertes. Cette architecture permet une couverture large et rapide tout en limitant les redondances.

Chapitre 4 — La boîte à outils des agents

4.1 Vue d'ensemble

Les agents disposent d'un ensemble d'outils couvrant l'ensemble du cycle offensif. Chaque outil est exposé au LLM avec un schéma d'appel, et exécuté soit côté hôte, soit dans le bac à sable.

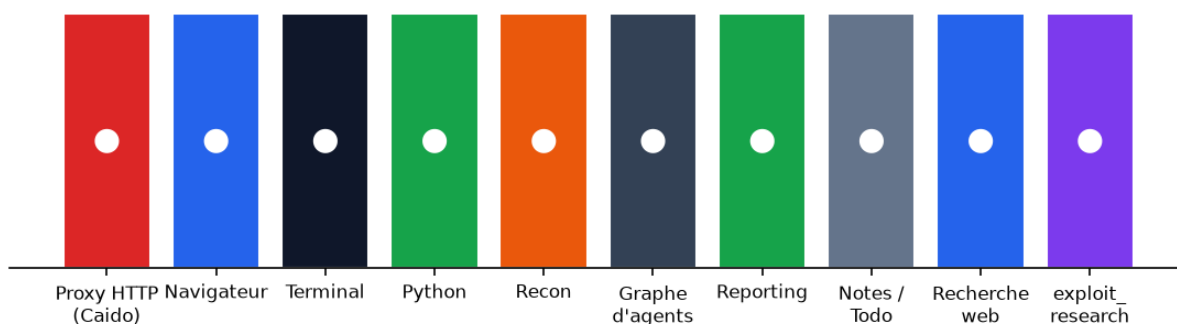


Figure 4.1 — Catégories d'outils des agents.

Outil	Rôle
proxy (Caïdo)	Proxy HTTP complet : interception et manipulation requêtes/réponses
browser	Automatisation d'un navigateur multi-onglets (XSS, CSRF, flux d'auth)
terminal	Shells interactifs pour l'exécution de commandes
python	Runtime Python pour développer et valider des exploits
agents_graph	Création et coordination des sous-agents
reporting	Documentation structurée des vulnérabilités et rapport final
notes / todo	Gestion des découvertes et des tâches
thinking	Raisonnement explicite (chaîne de pensée)
web_search	Recherche d'informations (CVE, techniques, PoC)
file_edit	Lecture/édition de fichiers (analyse de code)
finish	Clôture d'un agent ou de la campagne
exploit_research	(Extension) RAG + Metasploit, recommandation « module

	d'abord »
--	-----------

Tableau 4.1 — Boîte à outils des agents Strix.

Chapitre 5 — Base de connaissances (skills)

5.1 Principe des skills

Les « skills » sont des paquets de connaissances spécialisées, injectés dynamiquement dans le prompt d'un agent selon sa tâche (jusqu'à cinq par agent). Ils apportent techniques avancées, exemples de charges utiles et méthodes de validation, allant au-delà des connaissances de base du modèle.

5.2 Catégories de skills

Catégorie	Contenu (extrait)
vulnerabilities (17)	SQLi, XSS, XXE, SSRF, RCE, IDOR, CSRF, JWT/auth, BFLA, business logic, race conditions, mass assignment, open redirect, path traversal/LFI/RFI, file uploads, information disclosure, subdomain takeover
frameworks (2)	Techniques spécifiques à des frameworks web
technologies (2)	Services tiers (ex. Supabase, Firebase, passerelles de paiement)
protocols (1)	Tests propres à des protocoles (GraphQL, WebSocket, OAuth...)
cloud	Sécurité des fournisseurs cloud (AWS, Azure, GCP, Kubernetes)
reconnaissance	Collecte d'informations et cartographie de la surface d'attaque
coordination (1)	Règles d'orchestration de l'agent racine
scan_modes (3)	Modes quick / standard / deep
custom (1)	Skills communautaires / spécifiques

Tableau 5.1 — Catégories de skills de Strix.

5.3 Classes de vulnérabilités couvertes

Strix cible un large spectre de vulnérabilités, validées par preuve de concept.

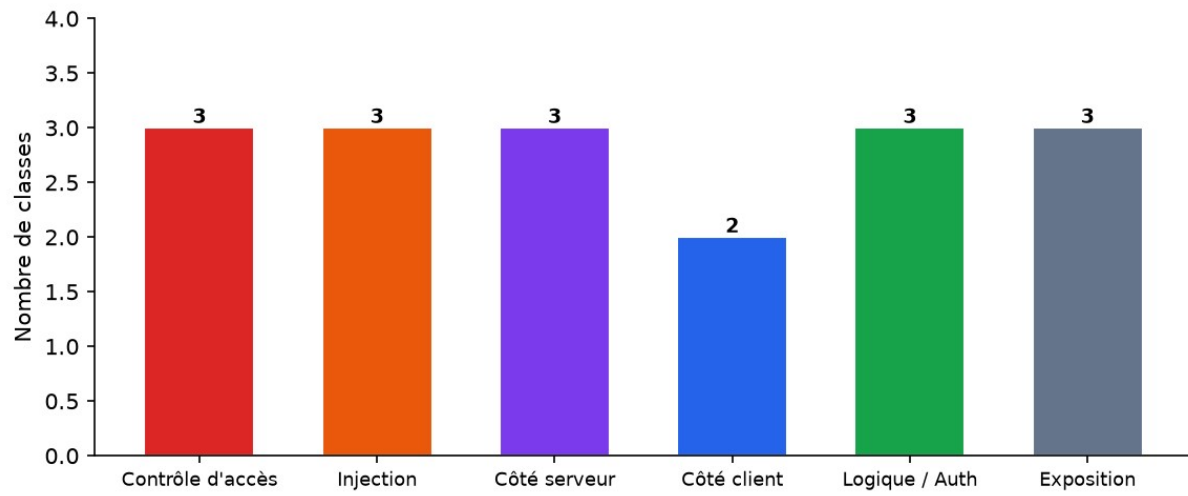


Figure 5.1 — Classes de vulnérabilités couvertes (regroupées par famille).

Chapitre 6 — Arsenal offensif et détection

6.1 Outils du bac à sable

L'image Kali du bac à sable embarque un arsenal d'outils de sécurité éprouvés, invoqués par les agents via le terminal ou des intégrations dédiées.

Outil	Usage principal
nmap	Découverte réseau et détection de services/versions
nuclei	Détection de vulnérabilités par templates
sqlmap	Détection et exploitation d'injections SQL
ffuf	Fuzzing de répertoires et de paramètres
httpx / katana	Sondage HTTP et crawling
subfinder / naabu	Énumération de sous-domaines et scan de ports
wapiti	Scan de vulnérabilités web
Metasploit (extension)	Exploitation par modules éprouvés
Caido	Proxy d'interception HTTP

Tableau 6.1 — Principaux outils de l'arsenal Strix.

6.2 Validation par preuve de concept

La spécificité de Strix est de ne pas se contenter de détecter : chaque vulnérabilité candidate est confirmée par un PoC réel (requête, script, session), ce qui distingue l'approche des scanners statiques sujets aux faux positifs.

Chapitre 7 — Flux d'exécution et livrables

7.1 Déroulement d'une campagne

Un test suit un enchaînement de phases : à partir d'une ou plusieurs cibles, les agents conduisent la reconnaissance et la cartographie, l'évaluation des vulnérabilités, l'exploitation, puis la validation par PoC, avant la production du rapport et des pistes de correction.



Figure 7.1 — Flux d'exécution d'une campagne.

7.2 Modes de scan

Trois modes ajustent la profondeur : quick (contrôles rapides pour CI/CD), standard (tests de routine) et deep (revue de sécurité approfondie, mode par défaut).

7.3 Livrables

Les résultats sont persistés dans `strix_runs/<nom-de-run>` :

- **vulnerabilities.csv** — liste des vulnérabilités (id, titre, sévérité, horodatage).
- **vulnerabilities/<id>.md** — détail par vulnérabilité (description, remédiation, PoC).
- **run.json** — métadonnées de la campagne et télémétrie LLM (requêtes, jetons, coût).
- **strix.log** — journal d'exécution complet.
- **.state/agents.json** — état et arborescence des agents.
- **penetration_test_report.md** — rapport de synthèse.

Chapitre 8 — Installation, configuration et intégration

8.1 Prérequis et installation

Prérequis : Docker en cours d'exécution et une clé de fournisseur LLM. L'installation se fait via un script (curl) ou pipx (strix-agent).

```
pipx install strix-agent
export STRIX_LLM="openai/gpt-5"
export LLM_API_KEY="votre-cle"
strix --target ./app-directory
```

8.2 Options de la ligne de commande

Option	Description
-t / --target	Cible : URL, dépôt Git, dossier local, domaine ou IP (répétable)
--instruction	Instructions personnalisées (portée, identifiants, focus)
--instruction-file	Fichier d'instructions détaillées (règles d'engagement)
-n / --non-interactive	Mode headless (sans TUI), pour serveurs et CI/CD
-m / --scan-mode	Profondeur : quick standard deep
--config	Fichier de configuration JSON personnalisé

Tableau 8.1 — Options principales de la CLI Strix.

8.3 Intégration CI/CD et cloud

Strix s'intègre aux pipelines via GitHub Actions : un test de sécurité peut être déclenché à chaque pull request (mode quick), la commande retournant un code non nul lorsque des vulnérabilités sont trouvées, ce qui permet de bloquer le code à risque. Une offre hébergée (app.strix.ai) propose rapports, tableaux de bord partagés, intégrations et surveillance continue sans configuration locale.

Chapitre 9 — Forces, limites et positionnement

9.1 Forces

- Validation réelle par PoC, réduisant fortement les faux positifs.
- Architecture multi-agent scalable et isolation par bac à sable Docker.
- Arsenal offensif complet et base de connaissances spécialisée (skills).
- Agnosticisme LLM (LiteLLM) et intégration CI/CD native.
- Open-source (Apache 2.0), documenté et outillé pour les développeurs.

9.2 Limites et points d'attention

- Coût et variabilité liés aux appels LLM sur les cibles complexes.
- Fiabilité de l'exploitation dépendante de la qualité du modèle (d'où l'intérêt d'extensions comme l'intégration Metasploit + RAG).
- Nécessité d'un cadre d'autorisation strict (tests uniquement sur des cibles autorisées).
- Dépendance à Docker et à la disponibilité de l'image du bac à sable.

9.3 Positionnement

Strix se positionne comme une plateforme de pentest assisté par IA orientée développeurs et équipes de sécurité, à mi-chemin entre les scanners automatisés (rapides mais bruités) et le pentest manuel (précis mais lent). Sa validation par PoC et son orchestration multi-agent en font une référence de la catégorie émergente des « hackers IA » open-source.

Conclusion

Strix illustre le potentiel des architectures agentiques appliquées à la sécurité offensive : en confiant à des équipes d'agents IA la conduite autonome d'un test d'intrusion, exécuté dans un bac à sable isolé et validé par des preuves de concept, le projet réconcilie rapidité, couverture et fiabilité. Son architecture modulaire — orchestrateur, bac à sable, LLM agnostique, boîte à outils et base de connaissances — en fait une base solide et extensible, comme le montrent les intégrations possibles (Metasploit, RAG).

Publié sous licence Apache 2.0 et intégrable en CI/CD, Strix constitue aujourd'hui l'une des références open-source du pentest assisté par IA, tout en rappelant les précautions d'usage propres à tout outil offensif : n'opérer que sur des cibles explicitement autorisées.

Références & Webographie

- [1] Dépôt officiel Strix — <https://github.com/usestrix/strix>
- [2] Documentation Strix — <https://docs.strix.ai>
- [3] Site officiel / offre cloud — <https://strix.ai> (app.strix.ai)
- [4] Paquet PyPI — strix-agent : <https://pypi.org/project/strix-agent/>
- [5] LiteLLM — passerelle multi-fournisseurs LLM. <https://docs.litellm.ai>
- [6] Metasploit Framework — <https://www.metasploit.com>
- [7] OWASP — Top 10 et Web Security Testing Guide. <https://owasp.org>
- [8] MITRE — CVE. <https://cve.mitre.org>
- [9] Caido — proxy d'interception HTTP. <https://caido.io>
- [10] Textual — framework TUI Python. <https://textual.textualize.io>